

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Червиченко Иван Денисович
2 курс, группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных DataFrame.

Ссылка на репозиторий: <https://github.com/progwar/ailab5>

Ход работы:

1. Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирования Python.

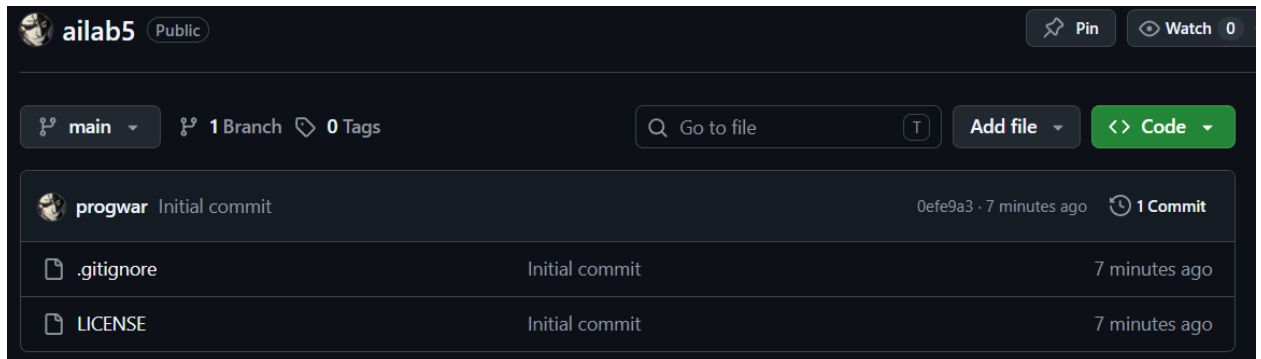


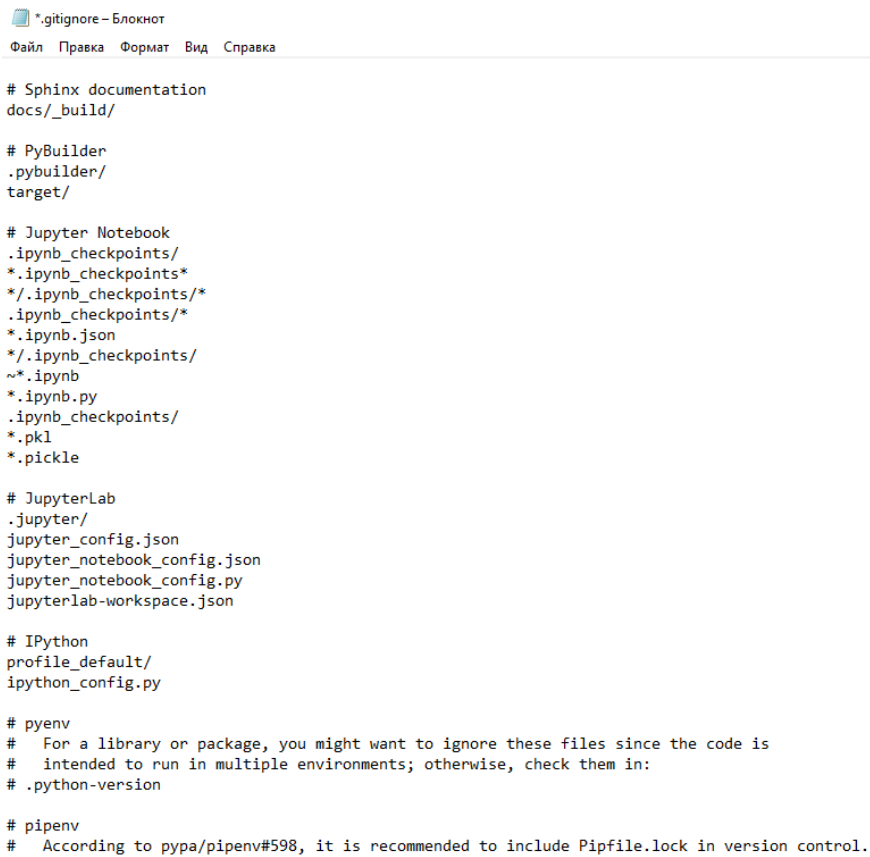
Рисунок 1. Созданный репозиторий

2. Репозиторий был клонирован на рабочий компьютер.

```
PS D:\github> git clone "https://github.com/progwar/ailab5.git"
Cloning into 'ailab5'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
```

Рисунок 2. Клонирование репозитория

2. Файл .gitignore был дополнен необходимыми правилами.

The image shows a screenshot of a text editor window titled ".gitignore - Блокнот". The menu bar includes "Файл", "Правка", "Формат", "Вид", and "Справка". The main text area contains a .gitignore file with the following content:

```
# Sphinx documentation
docs/_build/

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints/
*.ipynb_checkpoints*
*/.ipynb_checkpoints/*
.ipynb_checkpoints/*
*.ipynb.json
*/.ipynb_checkpoints/
~*.ipynb
*.ipynb.py
.ipynb_checkpoints/
*.pkl
*.pickle

# JupyterLab
.jupyter/
jupyter_config.json
jupyter_notebook_config.json
jupyter_notebook_config.py
jupyterlab-workspace.json

# IPython
profile_default/
ipython_config.py

# pyenv
# For a library or package, you might want to ignore these files since the code is
# intended to run in multiple environments; otherwise, check them in:
# .python-version

# pipenv
# According to pya/pipenv#598, it is recommended to include Pipfile.lock in version control.
```

Рисунок 3. Дополненный .gitignore файл

4. Были проработаны примеры лабораторной работы.

```
[2]: import pandas as pd
```

```
[2]: data={
      "Возраст": [25, 30, 22],
      "Город": ["Москва", "СПб", "Казань"]
    }
    df = pd.DataFrame(data, index=["Анна", "Иван", "Ольга"])
    print(df)
```

	Возраст	Город
Анна	25	Москва
Иван	30	СПб
Ольга	22	Казань

```
[3]: print(df["Возраст"])
```

Анна	25
Иван	30
Ольга	22

Name: Возраст, dtype: int64

Преобразование Series в DataFrame

```
[4]: s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
    df = s.to_frame(name='Значение')
    print(df)
```

	Значение
a	10
b	20
c	30

```
[5]: s = df['Значение']
    print(s)
```

a	10
b	20
c	30

Name: Значение, dtype: int64

Рисунок 4. Примеры (1)

```
[8]: data={
      "Имя":["Анна","Иван","Ольга"],
      "Возраст":[25,30,22],
      "Город":["Москва","СПб","Казань"]
    }
    df = pd.DataFrame(data)
    print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[9]: ages = pd.Series([25,30,22], index=["a","b","c"])
    cities = pd.Series(["Москва","СПб","Казань"], index=["a","b","c"])
    df= pd.DataFrame({"Возраст": ages, "Город": cities})
    print(df)
```

	Возраст	Город
a	25	Москва
b	30	СПб
c	22	Казань

```
[10]: data = [
        ["Anna",25,"Moscow"],
        ["Ivan",30,"SPb"],
        ["Olga",22,"Kazan"]
      ]
    df = pd.DataFrame(data, columns = ["Name","Age","City"])
    print(df)
```

	Name	Age	City
0	Anna	25	Moscow
1	Ivan	30	SPb
2	Olga	22	Kazan

```
[13]: data = [
        {"Name": "Anna","Age":25,"City":"Moscow"},
        {"Name":"Ivan","Age":30},
        {"Name":"Olga","Age":22,"City":"Kazan"}
      ]
    df = pd.DataFrame(data)
    print(df)
```

	Name	Age	City
0	Anna	25	Moscow
1	Ivan	30	NaN
2	Olga	22	Kazan

Рисунок 5. Примеры (2)

```

: import numpy as np
  data = np.array([
      ["Anna", 25, "Moskow"],
      ["Ivan", 30, "SPb"],
      ["Olga", 22, "Kazan"]
  ])
  df = pd.DataFrame(data, columns=["Name", "Age", "City"])
  print(df)

      Name Age   City
0  Anna  25  Moskow
1  Ivan  30    SPb
2  Olga  22   Kazan

```

```

: numeric_data = np.array([
    [25, 65.5],
    [30, 78.2],
    [22, 54.3]
])
  df = pd.DataFrame(numeric_data, columns=["Age", "Weight"])
  print(df)

      Age  Weight
0  25.0   65.5
1  30.0   78.2
2  22.0   54.3

```

```

: df = pd.DataFrame(columns=["Name", "Age", "City"])
  print(df)

Empty DataFrame
Columns: [Name, Age, City]
Index: []

```

```

: df.loc[0] = ["Anna", 25, "Moskow"]
  df.loc[1] = ["Ivan", 30, "SPb"]
  print(df)

      Name Age   City
0  Anna  25  Moskow
1  Ivan  30    SPb

```

Рисунок 6. Примеры (3)

00	; 13	00	; 69	00	; 3	00	; 89	00	;;;;;;;;;	
0	00	; 15	00	; 100	00	; 50	00	; 19	00	;;;;;;;;;
1	00	; 56	00	; 23	00	; 1	00	; 92	00	;;;;;;;;;
2	00	; 25	00	; 74	00	; 54	00	; 72	00	;;;;;;;;;
3	00	; 43	00	; 76	00	; 7	00	; 79	00	;;;;;;;;;
4	00	; 98	00	; 20	00	; 70	00	; 96	00	;;;;;;;;;

	Unnamed: 19	Unnamed: 20	Unnamed: 21	Unnamed: 22
1				
2.0	NaN	NaN	NaN	NaN
3.0	NaN	NaN	NaN	NaN
4.0	NaN	NaN	NaN	NaN
5.0	NaN	NaN	NaN	NaN
6.0	NaN	NaN	NaN	NaN

Рисунок 7. Примеры (4)

```
df.to_csv("output.csv", index=False)
```

```
import sqlite3
conn = sqlite3.connect("database.db")

conn.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER,
        name TEXT,
        email TEXT
    )
''')
conn.execute("INSERT INTO users VALUES (1, 'Alice', 'alice@example.com')")
conn.execute("INSERT INTO users VALUES (2, 'Bob', 'bob@example.com')")
conn.commit()
```

```
df = pd.read_sql("SELECT * FROM users", conn)
print(df.head())
```

	id	name	email
0	1	Alice	alice@example.com
1	2	Bob	bob@example.com

```
df.to_sql("users", conn, if_exists="replace", index=False)
```

2

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Сергей"],
    "Возраст": [25, 30, 22, 40, 35, 28],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург", "Сочи"]
}
df = pd.DataFrame(data)
print(df.head(3))
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
print(df.tail(2))
```

	Имя	Возраст	Город
4	Мария	35	Екатеринбург
5	Сергей	28	Сочи

Рисунок 8. Примеры (5)


```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Имя      6 non-null     str
1   Возраст  6 non-null     int64
2   Город    6 non-null     str
dtypes: int64(1), str(2)
memory usage: 276.0 bytes
```

```
print(df.describe())
```

	Возраст
count	6.00000
mean	30.00000
std	6.60303
min	22.00000
25%	25.75000
50%	29.00000
75%	33.75000
max	40.00000

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр"],
    "Возраст": [25, 30, 22, 40],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
}
```

```
df = pd.DataFrame(data, index=["a", "b", "c", "d"])
print(df)
```

	Имя	Возраст	Город
a	Анна	25	Москва
b	Иван	30	СПб
c	Ольга	22	Казань
d	Петр	40	Новосибирск

```
print(df.loc["c", "Город"])
```

Казань

```
print(df.loc[["a", "c"]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

Рисунок 9. Примеры (6)

```
print(df.loc[:,["Имя","Город"]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

```
print(df.loc[df["Возраст"]>25])
```

	Имя	Возраст	Город
b	Иван	30	СПб
d	Петр	40	Новосибирск

```
print(df.iloc[1])
```

	Имя	Возраст	Город
b	Иван	30	СПб

Name: b, dtype: object

```
print(df.iloc[2,2])
```

Казань

```
print(df.iloc[[0,2]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

```
print(df.iloc[1:3])
```

	Имя	Возраст	Город
b	Иван	30	СПб
c	Ольга	22	Казань

```
print(df.iloc[:,[0,2]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

```
print(df.at["c","Город"])
```

Казань

```
print(df.iat[2,2])
```

Казань

Рисунок 10. Примеры (7)

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22]
}

df = pd.DataFrame(data)
df["Город"] = ["Москва", "СПб", "Казань"]
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
df["Страна"] = "Россия"
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия

```
df["Возраст в месяцах"] = df["Возраст"] * 12
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах
0	Анна	25	Москва	Россия	300
1	Иван	30	СПб	Россия	360
2	Ольга	22	Казань	Россия	264

```
df = df.assign(Зарплата=[50000, 60000, 45000])
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах	Зарплата
0	Анна	25	Москва	Россия	300	50000
1	Иван	30	СПб	Россия	360	60000
2	Ольга	22	Казань	Россия	264	45000

```
df["Категория возраста"] = df["Возраст"].apply(lambda x: "Молодой" if x < 30 else "Взрослый")
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300	50000	
1	Иван	30	СПб	Россия	360	60000	
2	Ольга	22	Казань	Россия	264	45000	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой

Рисунок 11. Примеры (8)

```
df.loc[3]=["Петр",40,"Новосибирск","Россия"]
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия

```
new_data = pd.DataFrame([{"Имя":"Сергей", "Возраст":30, "Город":"Сочи", "Страна":"Россия"}])
df = pd.concat([df, new_data], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия

```
new_rows = pd.DataFrame([
    {"Имя": "Дмитрий", "Возраст": 31, "Город": "Самара", "Страна": "Россия"},
    {"Имя": "Елена", "Возраст": 27, "Город": "Воронеж", "Страна": "Россия"}
])

df = pd.concat([df, new_rows], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия
5	Дмитрий	31	Самара	Россия
6	Елена	27	Воронеж	Россия

```
df.loc["новый"]=["Артем",33,"Калуга","Россия"]
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия
5	Дмитрий	31	Самара	Россия
6	Елена	27	Воронеж	Россия
новый	Артем	33	Калуга	Россия

Рисунок 12. Примеры (9)

```
data = {
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"],
    "Зарплата": [50000, 60000, 45000]
}
df = pd.DataFrame(data)
df = df.drop(columns=["Зарплата"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
df = df.drop(columns=["Возраст", "Город"])
print(df)
```

	Имя
0	Анна
1	Иван
2	Ольга

```
del df["Имя"]
print(df)
```

	Возраст	Город
0	25	Москва
1	30	СПб
2	22	Казань

```
df["Имя"] = ["Анна", "Иван", "Ольга"]
удаленный_столбец = df.pop("Имя")
print(удаленный_столбец)
```

0	Анна
1	Иван
2	Ольга

Name: Имя, dtype: str

```
df = pd.DataFrame(data)
df = df.drop(index=1)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
2	Ольга	22	Казань	45000

Рисунок 13. Примеры (10)

```
df = pd.DataFrame(data)
df = df.drop(index=[0,2])
print(df)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

```
df = pd.DataFrame(data)
df = df[df["Возраст"]>25]
print(df)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

```
df = df.reset_index(drop=True)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Иван	30	СПб	60000

```
df = pd.DataFrame(data)
df.loc[3]=["Мария", None, "Самара", 55000]
df = df.dropna()
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000

```
df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Анна", "Ольга"],
    "Возраст": [25, 30, 25, 22]
})

df = df.drop_duplicates()
print(df)
```

	Имя	Возраст
0	Анна	25
1	Иван	30
3	Ольга	22

Рисунок 14. Примеры (11)

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}
```

```
df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
df_filt = df[df["Город"].isin(["Москва", "СПб"])]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
df_filt = df[~df["Город"].isin(["Москва", "СПб"])]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000

```
df_filt = df[df["Возраст"].between(25, 35)]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
df_filt = df[df["Возраст"].between(25, 35, inclusive="neither")]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

Рисунок 15. Примеры (12)

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", None],
    "Возраст": [25, 30, 22, None, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город
0	Анна	25.0	Москва
1	Иван	30.0	СПб
2	Ольга	22.0	Казань
3	Петр	NaN	Новосибирск
4	NaN	35.0	СПб

```
print(df.count())
```

```
Имя      4
Возраст  4
Город    5
dtype: int64
```

```
print(df["Город"].value_counts())
```

```
Город
СПб      2
Москва   1
Казань   1
Новосибирск  1
Name: count, dtype: int64
```

```
print(df["Город"].value_counts(sort=False))
```

```
Город
Москва   1
СПб      2
Казань   1
Новосибирск  1
Name: count, dtype: int64
```

```
print(df["Имя"].value_counts(dropna=False))
```

```
Имя
Анна    1
Иван    1
Ольга   1
Петр    1
NaN     1
Name: count, dtype: int64
```

Рисунок 16. Примеры (13)


```
print(df.nunique())
```

```
Имя      4
Возраст   4
Город     4
dtype: int64
```

```
print(df["Имя"].nunique(dropna=False))
```

```
5
```

```
print(df.isna().sum())
```

```
Имя      1
Возраст   1
Город     0
dtype: int64
```

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}
```

```
df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
df_sort = df.sort_values(by="Возраст")
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
df_sort = df.sort_values(by="Возраст", ascending=False)
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
1	Иван	30	СПб	60000
0	Анна	25	Москва	50000
2	Ольга	22	Казань	45000

Рисунок 17. Примеры (14)

```
df_sort = df.sort_values(by=["Возраст", "Зарплата"], ascending=[True,
                                                                    False])
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
df.loc[5] = ["Елена", None, "Самара", 55000]
df_sort = df.sort_values(by="Возраст", na_position="first")
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
5	Елена	None	Самара	55000
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
df_sort = df.sort_index()
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
5	Елена	None	Самара	55000

```
df_sort = df.sort_index(ascending=False)
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
5	Елена	None	Самара	55000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000
2	Ольга	22	Казань	45000
1	Иван	30	СПб	60000
0	Анна	25	Москва	50000

Рисунок 18. Примеры (15)

```
df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})

print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
from IPython.display import display
display(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Мария"],
    "Возраст": [25, 30, 22, 40, 35, 43],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб", "Новосибирск"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000, 90000]
}

df = pd.DataFrame(data)
df.head(3)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000

```
df.tail(3)
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
5	Мария	43	Новосибирск	90000

Рисунок 19. Примеры (16)

```

: pd.set_option("display.max_rows", 100)

: pd.set_option("display.max_columns", 50)

: pd.set_option("display.max_colwidth", None)

: pd.set_option("display.float_format", "{:.2f}".format)#отображение чисел с плавающей точкой

: df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})
df.style.set_properties(**{"background-color": "lightgray", "color": "black"})

html_table = df.to_html()
with open("table.html", "w", encoding="utf-8") as f:
    f.write(html_table)

    Имя  Возраст  Город
0  Анна      25  Москва
1  Иван      30   СПб
2  Ольга     22  Казань

: styled_df = df.style.set_properties(**{"background-color": "lightgray", "color": "blue"})
display(styled_df)

    Имя  Возраст  Город
0  Анна      25  Москва
1  Иван      30   СПб
2  Ольга     22  Казань

```

Рисунок 20. Примеры (17)

5. Были выполнены практические задания.

Таблица 1: Данные о сотрудниках

ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
1	Иван	25	Инженер	IT	60000	2
2	Ольга	30	Аналитик	Маркетинг	75000	5
3	Алексей	40	Менеджер	Продажи	90000	15
4	Мария	35	Программист	IT	80000	7
5	Сергей	28	Специалист	HR	50000	3
6	Анна	32	Разработчик	IT	85000	6
7	Дмитрий	45	HR	HR	48000	12
8	Елена	29	Маркетолог	Маркетинг	70000	4
9	Виктор	31	Юрист	Юридический	95000	10
10	Алиса	27	Дизайнер	Дизайн	62000	5
11	Павел	33	Администратор	Администрация	55000	7
12	Светлана	26	Тестирующий	Тестирование	67000	2
13	Роман	42	Финансист	Финансы	105000	20
14	Татьяна	37	Редактор	Редакция	72000	9
15	Николай	39	Логист	Логистика	75000	11
16	Валерия	24	SEO-специалист	SEO	64000	3
17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25
18	Юлия	45	Директор	Финансы	150000	20
19	Степан	41	Экономист	Экономика	98000	14
20	Василиса	38	Проект-менеджер	Продажи	88000	8

Рисунок 21. Данные о сотрудниках

Таблица 2: Данные о клиентах

ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
1	Иван	34	Москва	120000	Хорошая
2	Ольга	27	Санкт-Петербург	80000	Средняя
3	Алексей	45	Казань	150000	Плохая
4	Мария	38	Новосибирск	200000	Хорошая
5	Сергей	29	Екатеринбург	95000	Средняя
6	Анна	50	Воронеж	300000	Отличная
7	Дмитрий	31	Челябинск	140000	Средняя
8	Елена	40	Краснодар	175000	Хорошая
9	Виктор	28	Ростов-на-Дону	110000	Плохая

ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
10	Алиса	33	Уфа	98000	Средняя
11	Павел	46	Омск	250000	Хорошая
12	Светлана	37	Пермь	210000	Отличная
13	Роман	41	Тюмень	135000	Средняя
14	Татьяна	25	Саратов	155000	Хорошая
15	Николай	39	Самара	125000	Средняя
16	Валерия	42	Волгоград	180000	Плохая
17	Григорий	49	Барнаул	275000	Отличная
18	Юлия	50	Иркутск	320000	Хорошая
19	Степан	30	Хабаровск	105000	Средняя
20	Василиса	35	Томск	90000	Плохая

Рисунок 22. Данные о клиентах

Таблица 3: Данные с пропусками

ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
1	Иван	34.0	Москва	120000.0	Хорошая
2	Ольга	27.0	Санкт-Петербург		Средняя
3	Алексей		Казань	150000.0	Плохая
4	Мария	38.0		200000.0	Хорошая
5	Сергей	29.0	Екатеринбург		
6		50.0	Воронеж	300000.0	Отличная
7	Дмитрий	31.0		140000.0	Средняя
8	Елена	40.0	Краснодар	175000.0	Хорошая
9	Виктор		Ростов-на-Дону	110000.0	
10	Алиса	33.0	Уфа		Средняя
11	Павел	46.0	Омск	250000.0	Хорошая
12		37.0	Пермь	210000.0	Отличная
13	Роман	41.0		135000.0	Средняя
14	Татьяна	25.0	Саратов	155000.0	
15	Николай		Самара	125000.0	Средняя
16	Валерия	42.0			Плохая
17	Григорий	49.0	Барнаул	275000.0	Отличная
18	Юлия		Иркутск	320000.0	Хорошая
19	Степан	30.0	Хабаровск	105000.0	Средняя
20	Василиса	35.0	Томск	90000.0	Плохая

Рисунок 23. Данные с пропусками

1. Создание DataFrame разными способами

1. Создайте DataFrame из словаря списков с данными о пяти сотрудниках (возьмите из таблицы 1).
2. Создайте DataFrame из списка словарей с теми же данными.
3. Создайте DataFrame из массива NumPy, заполненного случайными числами от 20 до 60 (для возраста сотрудников).
4. Проверьте типы данных в каждом столбце с помощью `.info()`.

Рисунок 24. 1 задание

```

import pandas as pd
import numpy as np

data_dict = {
    'ID': [1, 2, 3, 4, 5],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей'],
    'Возраст': [25, 30, 40, 35, 28],
    'Должность': ['Инженер', 'Аналитик', 'Менеджер', 'Программист', 'Специалист'],
    'Отдел': ['IT', 'Маркетинг', 'Продажи', 'IT', 'HR'],
    'Зарплата': [60000, 75000, 90000, 80000, 50000],
    'Стаж работы': [2, 5, 15, 7, 3]
}

df_dict = pd.DataFrame(data_dict)
print("DataFrame из словаря списков:")
print(df_dict)

list_dict = [
    {'ID': 1, 'Имя': 'Иван', 'Возраст': 25, 'Должность': 'Инженер', 'Отдел': 'IT', 'Зарплата': 60000, 'Стаж работы': 2},
    {'ID': 2, 'Имя': 'Ольга', 'Возраст': 30, 'Должность': 'Аналитик', 'Отдел': 'Маркетинг', 'Зарплата': 75000, 'Стаж работы': 5},
    {'ID': 3, 'Имя': 'Алексей', 'Возраст': 40, 'Должность': 'Менеджер', 'Отдел': 'Продажи', 'Зарплата': 90000, 'Стаж работы': 15},
    {'ID': 4, 'Имя': 'Мария', 'Возраст': 35, 'Должность': 'Программист', 'Отдел': 'IT', 'Зарплата': 80000, 'Стаж работы': 7},
    {'ID': 5, 'Имя': 'Сергей', 'Возраст': 28, 'Должность': 'Специалист', 'Отдел': 'HR', 'Зарплата': 50000, 'Стаж работы': 3}
]

df_list = pd.DataFrame(list_dict)
print("DataFrame из списка словарей:")
print(df_list)

age_array = np.random.randint(20, 61, size=(10, 1))
df_numpy = pd.DataFrame(age_array, columns=['Возраст'])
print("DataFrame из массива NumPy (случайный возраст):")
print(df_numpy)
print("\n")

print("Информация о типах данных для df_dict:")
df_dict.info()
print("\n")

print("Информация о типах данных для df_list:")
df_list.info()
print("\n")

print("Информация о типах данных для df_numpy:")
df_numpy.info()

```

Рисунок 25. Код 1 задания


```
DataFrame из словаря списков:
  ID  Имя  Возраст  Должность  Отдел  Зарплата  Стаж работы
0  1  Иван    25    Инженер    IT      60000        2
1  2  Ольга    30    Аналитик  Маркетинг  75000        5
2  3  Алексей   40    Менеджер  Продажи   90000       15
3  4  Мария    35  Программист    IT      80000        7
4  5  Сергей    28    Специалист  HR      50000        3

DataFrame из списка словарей:
  ID  Имя  Возраст  Должность  Отдел  Зарплата  Стаж работы
0  1  Иван    25    Инженер    IT      60000        2
1  2  Ольга    30    Аналитик  Маркетинг  75000        5
2  3  Алексей   40    Менеджер  Продажи   90000       15
3  4  Мария    35  Программист    IT      80000        7
4  5  Сергей    28    Специалист  HR      50000        3

DataFrame из массива NumPy (случайный возраст):
  Возраст
0      58
1      33
2      41
3      36
4      30
5      41
6      54
7      49
8      35
9      52
```

Информация о типах данных для df_dict:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   ID          5 non-null      int64
1   Имя         5 non-null      object
2   Возраст     5 non-null      int64
3   Должность   5 non-null      object
4   Отдел       5 non-null      object
5   Зарплата    5 non-null      int64
6   Стаж работы  5 non-null      int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
```

Информация о типах данных для df_list:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   ID          5 non-null      int64
1   Имя         5 non-null      object
2   Возраст     5 non-null      int64
3   Должность   5 non-null      object
4   Отдел       5 non-null      object
5   Зарплата    5 non-null      int64
6   Стаж работы  5 non-null      int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
```

Информация о типах данных для df_numpy:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 1 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Возраст     10 non-null     int32
dtypes: int32(1)
memory usage: 172.0 bytes
```

Рисунок 26. Результат 1 задания

2. Чтение данных из файлов (CSV, Excel, JSON)

Используйте предоставленные таблицы и сохраните их в файлы перед чтением.

1. Сохраните **Таблицу 1** в **CSV** и затем загрузите ее обратно в **DataFrame**.
2. Запишите **Таблицу 2** в **Excel** (`data.xlsx`, лист "Клиенты") и прочитайте ее в **DataFrame**.
3. Экпортируйте **Таблицу 1** в формат **JSON** и затем прочитайте ее обратно в **DataFrame**.

Выведите первые 5 строк загруженных **DataFrame**.

Рисунок 27. 2 задание

```
import pandas as pd
import numpy as np
import json

data_table1 = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    'Должность': ['Инженер', 'Аналитик', 'Менеджер', 'Программист', 'Специалист', 'Разработчик', 'HR', 'Маркетолог', 'Юрист', 'Дизайнер',
                  'Администратор', 'Тестировщик', 'Финансист', 'Редактор', 'Логист', 'SEO-специалист', 'Бухгалтер', 'Директор', 'Экономист', 'Проект-менеджер'],
    'Отдел': ['IT', 'Маркетинг', 'Продажи', 'IT', 'HR', 'IT', 'HR', 'Маркетинг', 'Юридический', 'Дизайн',
              'Администрация', 'Тестирование', 'Финансы', 'Редакция', 'Логистика', 'SEO', 'Бухгалтерия', 'Финансы', 'Экономика', 'Продажи'],
    'Зарплата': [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 72000, 75000, 64000, 110000, 150000, 98000, 88000],
    'Стаж работы': [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
}

df_table1 = pd.DataFrame(data_table1)

data_table2 = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    'Город': ['Москва', 'Санкт-Петербург', 'Казань', 'Новосибирск', 'Екатеринбург', 'Воронеж', 'Челябинск', 'Краснодар', 'Ростов-на-Дону', 'Уфа',
              'Омск', 'Пермь', 'Тюмень', 'Саратов', 'Самара', 'Волгоград', 'Барнаул', 'Иркутск', 'Хабаровск', 'Томск'],
    'Баланс на счете': [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 90000,
                       250000, 210000, 135000, 155000, 125000, 180000, 275000, 320000, 105000, 90000],
    'Кредитная история': ['Хорошая', 'Средняя', 'Плохая', 'Хорошая', 'Средняя', 'Отличная', 'Средняя', 'Хорошая', 'Плохая', 'Средняя',
                           'Хорошая', 'Отличная', 'Средняя', 'Хорошая', 'Средняя', 'Плохая', 'Отличная', 'Хорошая', 'Средняя', 'Плохая']
}

df_table2 = pd.DataFrame(data_table2)

df_table1.to_csv('tab1.csv', index=False)
print("Таблица 1 сохранена в 'tab1.csv'")

df_csv_load = pd.read_csv('tab1.csv')
print("Загружена из 'tab1.csv'")
print("\n Первые 5 строк загруженного DataFrame из CSV:")
print(df_csv_load.head())
print("\n")

df_table2.to_excel('tab2.xlsx', sheet_name='Клиенты', index=False)
print("Таблица 2 сохранена в 'tab2.xlsx', лист 'Клиенты'")

df_excel_load = pd.read_excel('tab2.xlsx', sheet_name='Клиенты')
print("Загружена из 'tab2.xlsx', лист 'Клиенты'")
print("\n Первые 5 строк загруженного DataFrame из Excel:")
print(df_excel_load.head())
print("\n")

df_table1.to_json('tab1js.json', orient='records', indent=2)
print("Таблица 1 сохранена в 'tab1js.json'")

df_json_load = pd.read_json('tab1js.json')
print("Загружена из 'tab1js.json'")
print("\n Первые 5 строк загруженного DataFrame из JSON:")
print(df_json_load.head())
```

Рисунок 28. Код 2 задания

Таблица 1 сохранена в 'tab1.csv'
Загружена из 'tab1.csv'

Первые 5 строк загруженного DataFrame из CSV:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Таблица 2 сохранена в 'tab2.xlsx', лист 'Клиенты'
Загружена из 'tab2.xlsx', лист 'Клиенты'

Первые 5 строк загруженного DataFrame из Excel:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
4	5	Сергей	29	Екатеринбург	95000	Средняя

Таблица 1 сохранена в 'tab1js.json'
Загружена из 'tab1js.json'

Первые 5 строк загруженного DataFrame из JSON:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 29. Результат 2 задания

3. Доступ к данным (`.loc` , `.iloc` , `.at` , `.iat`)

Используя **Таблицу 1**:

1. Получите информацию о сотруднике с **ID = 5** с помощью `.loc[]` .
2. Выведите возраст **третьего сотрудника** в таблице с `.iloc[]` .
3. Выведите название **отдела** для сотрудника "Мария" с `.at[]` .
4. Выведите **зарплату** сотрудника, находящегося в **четвертой строке** и **пятом столбце**, используя `.iat[]` .

Объясните разницу между `.loc[]` , `.iloc[]` , `.at[]` , `.iat[]` .

Рисунок 30. 3 задание

```

import pandas as pd
import numpy as np

data_table1 = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    'Должность': ['Инженер', 'Аналитик', 'Менеджер', 'Программист', 'Специалист', 'Разработчик', 'HR', 'Маркетолог', 'Юрист', 'Дизайнер',
                  'Администратор', 'Тестировщик', 'Финансист', 'Редактор', 'Логист', 'SEO-специалист', 'Бухгалтер', 'Директор', 'Экономист', 'Проект-менеджер'],
    'Отдел': ['IT', 'Маркетинг', 'Продажи', 'IT', 'HR', 'IT', 'HR', 'Маркетинг', 'Юридический', 'Дизайн',
              'Администрация', 'Тестирование', 'Финансы', 'Редакция', 'Логистика', 'SEO', 'Бухгалтерия', 'Финансы', 'Экономика', 'Продажи'],
    'Зарплата': [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 72000, 75000, 64000, 110000, 150000, 90000, 88000],
    'Стаж работы': [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
}

df = pd.DataFrame(data_table1)

print("1. Сотрудник с ID=5 (.loc[]):")
print(df.loc[df['ID'] == 5])
print("\n")

print("2. Возраст третьего сотрудника (.iloc[]):")
print(df.iloc[2, 2])
print("\n")

print("3. Отдел Марии (.at[]):")
index_maria = df[df['Имя'] == 'Мария'].index[0]
print(df.at[index_maria, 'Отдел'])
print("\n")

print("4. Зарплата (4-я строка, 5-й столбец) (.iat[]):")
print(df.iat[3, 5])
print("\n")

print("Разница методов:")
print("""
.loc[] - по названиям (меткам) -> df.loc[2, 'Имя']
.iloc[] - по позициям (номерам) -> df.iloc[2, 0]
.at[] - по названиям, но быстрее (одно значение)
.iat[] - по позициям (одно значение)""")

1. Сотрудник с ID=5 (.loc[]):
   ID  Имя  Возраст  Должность  Отдел  Зарплата  Стаж работы
4    5  Сергей    28  Специалист   HR    50000         3

2. Возраст третьего сотрудника (.iloc[]):
40

3. Отдел Марии (.at[]):
IT

4. Зарплата (4-я строка, 5-й столбец) (.iat[]):
80000

Разница методов:

.loc[] - по названиям (меткам) -> df.loc[2, 'Имя']
.iloc[] - по позициям (номерам) -> df.iloc[2, 0]
.at[] - по названиям, но быстрее (одно значение)
.iat[] - по позициям (одно значение)

```

Рисунок 31. Выполнение 3 задания

4. Добавление новых столбцов и строк

Используя **Таблицу 1**:

1. Добавьте новый столбец "Категория зарплаты" :

- "Низкая" – если $\text{Зарплата} < 60000$.

- "Средняя" – если $60000 \leq \text{Зарплата} < 100000$.

- "Высокая" – если $\text{Зарплата} \geq 100000$.

2. Добавьте нового сотрудника с данными:



```
1 df.loc[21] = ["Антон", 32, "Разработчик", "IT", 85000, 6]
```

3. Добавьте **двух новых сотрудников** одновременно, используя `pd.concat()` .

Выведите обновленный `DataFrame` .

Рисунок 32. 4 задание

```

import pandas as pd

df = pd.DataFrame({
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    'Должность': ['Инженер', 'Аналитик', 'Менеджер', 'Программист', 'Специалист', 'Разработчик', 'HR', 'Маркетолог', 'Юрист', 'Дизайнер',
                  'Администратор', 'Тестировщик', 'Финансист', 'Редактор', 'Логист', 'SEO-специалист', 'Бухгалтер', 'Директор', 'Экономист', 'Проект-менеджер'],
    'Отдел': ['IT', 'Маркетинг', 'Продажи', 'IT', 'HR', 'IT', 'HR', 'Маркетинг', 'Юридический', 'Дизайн',
              'Администрация', 'Тестирование', 'Финансы', 'Редакция', 'Логистика', 'SEO', 'Бухгалтерия', 'Финансы', 'Экономика', 'Продажи'],
    'Зарплата': [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 72000, 75000, 64000, 110000, 150000, 98000, 88000],
    'Стаж работы': [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
})

def category_zarp(zarp):
    if zarp < 60000:
        return "Низкая"
    elif zarp < 100000:
        return "Средняя"
    else:
        return "Высокая"

df['Категория зарплаты'] = df['Зарплата'].apply(category_zarp)

print("1. После добавления столбца 'Категория зарплаты':")
print(df)
print("\n")

df.loc[20] = [21, "Антон", 32, "Разработчик", "IT", 85000, 6, "Средняя"]

print("2. После добавления сотрудника Антона (.loc[]):")
print(df)
print("\n")

new_workers = pd.DataFrame({
    'ID': [22, 23],
    'Имя': ['Елена', 'Дмитрий'],
    'Возраст': [29, 45],
    'Должность': ['Маркетолог', 'HR-специалист'],
    'Отдел': ['Маркетинг', 'HR'],
    'Зарплата': [70000, 48000],
    'Стаж работы': [4, 12],
    'Категория зарплаты': ['Средняя', 'Низкая']
})

df = pd.concat([df, new_workers], ignore_index=True)

print("3. После добавления двух сотрудников (pd.concat()):")
print(df)
print("\n")

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', None)

```

Рисунок 33. Код 4 задания

1. После добавления столбца 'Категория зарплаты':

ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты	
0	1	Иван	25	Инженер	IT	60000	2	Средняя
1	2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
2	3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
3	4	Мария	35	Программист	IT	80000	7	Средняя
4	5	Сергей	28	Специалист	HR	50000	3	Низкая
5	6	Анна	32	Разработчик	IT	85000	6	Средняя
6	7	Дмитрий	45	HR	HR	48000	12	Низкая
7	8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
8	9	Виктор	31	Юрист	Юридический	95000	10	Средняя
9	10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
10	11	Павел	33	Администратор	Администрация	55000	7	Низкая
11	12	Светлана	26	Тестировщик	Тестирование	67000	2	Средняя
12	13	Роман	42	Финансист	Финансы	105000	20	Высокая
13	14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
14	15	Николай	39	Логист	Логистика	75000	11	Средняя
15	16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
17	18	Илья	45	Директор	Финансы	150000	20	Высокая
18	19	Степан	41	Экономист	Экономика	98000	14	Средняя
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя

2. После добавления сотрудника Антона (.loc[]):

ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплат	
0	1	Иван	25	Инженер	IT	60000	2	Средняя
1	2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
2	3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
3	4	Мария	35	Программист	IT	80000	7	Средняя
4	5	Сергей	28	Специалист	HR	50000	3	Низкая
5	6	Анна	32	Разработчик	IT	85000	6	Средняя
6	7	Дмитрий	45	HR	HR	48000	12	Низкая
7	8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
8	9	Виктор	31	Юрист	Юридический	95000	10	Средняя
9	10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
10	11	Павел	33	Администратор	Администрация	55000	7	Низкая
11	12	Светлана	26	Тестировщик	Тестирование	67000	2	Средняя
12	13	Роман	42	Финансист	Финансы	105000	20	Высокая
13	14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
14	15	Николай	39	Логист	Логистика	75000	11	Средняя
15	16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
17	18	Юлия	45	Директор	Финансы	150000	20	Высокая
18	19	Степан	41	Экономист	Экономика	98000	14	Средняя
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя
20	21	Антон	32	Разработчик	IT	85000	6	Средняя

3. После добавления двух сотрудников (pd.concat()):

ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплат	
0	1	Иван	25	Инженер	IT	60000	2	Средняя
1	2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
2	3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
3	4	Мария	35	Программист	IT	80000	7	Средняя
4	5	Сергей	28	Специалист	HR	50000	3	Низкая
5	6	Анна	32	Разработчик	IT	85000	6	Средняя
6	7	Дмитрий	45	HR	HR	48000	12	Низкая
7	8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
8	9	Виктор	31	Юрист	Юридический	95000	10	Средняя
9	10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
10	11	Павел	33	Администратор	Администрация	55000	7	Низкая
11	12	Светлана	26	Тестировщик	Тестирование	67000	2	Средняя
12	13	Роман	42	Финансист	Финансы	105000	20	Высокая
13	14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
14	15	Николай	39	Логист	Логистика	75000	11	Средняя
15	16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
17	18	Илья	45	Директор	Финансы	150000	20	Высокая
18	19	Степан	41	Экономист	Экономика	98000	14	Средняя
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя
20	21	Антон	32	Разработчик	IT	85000	6	Средняя
21	22	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
22	23	Дмитрий	45	HR-специалист	HR	48000	12	Низкая

Рисунок 34. Результат 4 задания

5. Удаление строк и столбцов

Используя Таблицу 1:

1. Удалите столбец "Категория зарплаты" с `.drop()` .
2. Удалите строку с `ID = 10` .
3. Удалите все строки, где Стаж работы `< 3` лет .
4. Удалите все столбцы, кроме Имя , Должность , Зарплата .

Выведите `DataFrame` после каждого шага.

Рисунок 35. 5 задание

```
import pandas as pd

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', None)

data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    'Должность': ['Инженер', 'Аналитик', 'Менеджер', 'Программист', 'Специалист', 'Разработчик', 'HR', 'Маркетолог', 'Юрист', 'Дизайнер',
                  'Администратор', 'Тестировщик', 'Финансист', 'Редактор', 'Логист', 'SEO-специалист', 'Бухгалтер', 'Директор', 'Экономист', 'Проект-менеджер'],
    'Отдел': ['IT', 'Маркетинг', 'Продажи', 'IT', 'HR', 'IT', 'HR', 'Маркетинг', 'Юридический', 'Дизайн',
              'Администрация', 'Тестирование', 'Финансы', 'Редакция', 'Логистика', 'SEO', 'Бухгалтерия', 'Финансы', 'Экономика', 'Продажи'],
    'Зарплата': [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 72000, 75000, 64000, 110000, 150000, 98000, 88000],
    'Стаж работы': [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
}

df = pd.DataFrame(data)

def category_salary(salary):
    if salary < 60000:
        return "Низкая"
    elif salary < 100000:
        return "Средняя"
    else:
        return "Высокая"

df['Категория зарплаты'] = df['Зарплата'].apply(category_salary)

print("Исходный DataFrame:")
print(df)
print("\n")

print("1. Удаление столбца 'Категория зарплаты' с .drop():")
df = df.drop(columns=["Категория зарплаты"])
print(df)
print("\n")

print("2. Удаление строки с ID = 10:")
index_drop = df[df['ID'] == 10].index[0]
df = df.drop(index=index_drop)
print(df)
print("\n")

print("3. Удаление строк со стажем работы < 3 лет:")
before = len(df)
df = df[df['Стаж работы'] >= 3]
removed = before - len(df)
print(df)
print("\n")

print("-*80")
print("4. Оставляем только столбцы: Имя, Должность, Зарплата:")
df = df[['Имя', 'Должность', 'Зарплата']]
print(df)
```

Рисунок 36. Код 5 задания

Исходный DataFrame:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы	Категория зарплаты
0	1	Иван	25	Инженер	IT	60000	2	Средняя
1	2	Ольга	30	Аналитик	Маркетинг	75000	5	Средняя
2	3	Алексей	40	Менеджер	Продажи	90000	15	Средняя
3	4	Мария	35	Программист	IT	80000	7	Средняя
4	5	Сергей	28	Специалист	HR	50000	3	Низкая
5	6	Анна	32	Разработчик	IT	85000	6	Средняя
6	7	Дмитрий	45	HR	HR	48000	12	Низкая
7	8	Елена	29	Маркетолог	Маркетинг	70000	4	Средняя
8	9	Виктор	31	Юрист	Юридический	95000	10	Средняя
9	10	Алиса	27	Дизайнер	Дизайн	62000	5	Средняя
10	11	Павел	33	Администратор	Администрация	55000	7	Низкая
11	12	Светлана	26	Тестирующий	Тестирование	67000	2	Средняя
12	13	Роман	42	Финансист	Финансы	105000	20	Высокая
13	14	Татьяна	37	Редактор	Редакция	72000	9	Средняя
14	15	Николай	39	Логист	Логистика	75000	11	Средняя
15	16	Валерия	24	SEO-специалист	SEO	64000	3	Средняя
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25	Высокая
17	18	Юлия	45	Директор	Финансы	150000	20	Высокая
18	19	Степан	41	Экономист	Экономика	98000	14	Средняя
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8	Средняя

1. Удаление столбца 'Категория зарплаты' с .drop():

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3
5	6	Анна	32	Разработчик	IT	85000	6
6	7	Дмитрий	45	HR	HR	48000	12
7	8	Елена	29	Маркетолог	Маркетинг	70000	4
8	9	Виктор	31	Юрист	Юридический	95000	10
9	10	Алиса	27	Дизайнер	Дизайн	62000	5
10	11	Павел	33	Администратор	Администрация	55000	7
11	12	Светлана	26	Тестирующий	Тестирование	67000	2
12	13	Роман	42	Финансист	Финансы	105000	20
13	14	Татьяна	37	Редактор	Редакция	72000	9
14	15	Николай	39	Логист	Логистика	75000	11
15	16	Валерия	24	SEO-специалист	SEO	64000	3
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25
17	18	Юлия	45	Директор	Финансы	150000	20
18	19	Степан	41	Экономист	Экономика	98000	14
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8

2. Удаление строки с ID = 10:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3
5	6	Анна	32	Разработчик	IT	85000	6
6	7	Дмитрий	45	HR	HR	48000	12
7	8	Елена	29	Маркетолог	Маркетинг	70000	4
8	9	Виктор	31	Юрист	Юридический	95000	10
10	11	Павел	33	Администратор	Администрация	55000	7
11	12	Светлана	26	Тестирующий	Тестирование	67000	2
12	13	Роман	42	Финансист	Финансы	105000	20
13	14	Татьяна	37	Редактор	Редакция	72000	9
14	15	Николай	39	Логист	Логистика	75000	11
15	16	Валерия	24	SEO-специалист	SEO	64000	3
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25
17	18	Юлия	45	Директор	Финансы	150000	20
18	19	Степан	41	Экономист	Экономика	98000	14
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8

Рисунок 37. Результат 5 задания (1)

3. Удаление строк со стажем работы < 3 лет:

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж работы
1	2	Ольга	38	Аналитик	Маркетинг	75000	5
2	3	Алексей	48	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3
5	6	Анна	32	Разработчик	IT	85000	6
6	7	Дмитрий	45	HR	HR	48000	12
7	8	Елена	29	Маркетолог	Маркетинг	70000	4
8	9	Виктор	31	Юрист	Юридический	95000	10
10	11	Павел	33	Администратор	Администрация	55000	7
12	13	Роман	42	Финансист	Финансы	105000	20
13	14	Татьяна	37	Редактор	Редакция	72000	9
14	15	Николай	39	Логист	Логистика	75000	11
15	16	Валерия	24	SEO-специалист	SEO	64000	3
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25
17	18	Юлия	45	Директор	Финансы	150000	20
18	19	Степан	41	Экономист	Экономика	98000	14
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8

4. Оставляем только столбцы: Имя, Должность, Зарплата:

	Имя	Должность	Зарплата
1	Ольга	Аналитик	75000
2	Алексей	Менеджер	90000
3	Мария	Программист	80000
4	Сергей	Специалист	50000
5	Анна	Разработчик	85000
6	Дмитрий	HR	48000
7	Елена	Маркетолог	70000
8	Виктор	Юрист	95000
10	Павел	Администратор	55000
12	Роман	Финансист	105000
13	Татьяна	Редактор	72000
14	Николай	Логист	75000
15	Валерия	SEO-специалист	64000
16	Григорий	Бухгалтер	110000
17	Юлия	Директор	150000
18	Степан	Экономист	98000
19	Василиса	Проект-менеджер	88000

Рисунок 38. Результат 5 задания (2)

6. Фильтрация данных (query , isin , between)

Используя Таблицу 2:

1. Выберите всех клиентов из "Москва" или "Санкт-Петербург" , используя `.isin()` .
2. Выберите клиентов, у которых **Баланс на счете** от 100000 до 250000, используя `.between()` .
3. Отфильтруйте клиентов, у которых "Кредитная история" "Хорошая" и "Баланс на счете" > 150000 , используя `.query()` .

Выведите результаты каждого фильтра.

Рисунок 39. Задание 6

```

import pandas as pd

pd.set_option('display.width', None)
pd.set_option('display.max_columns', None)

data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    'Город': ['Москва', 'Санкт-Петербург', 'Казань', 'Новосибирск', 'Екатеринбург', 'Воронеж', 'Челябинск', 'Краснодар', 'Ростов-на-Дону', 'Уфа',
              'Омск', 'Пермь', 'Тюмень', 'Саратов', 'Самара', 'Волгоград', 'Барнаул', 'Иркутск', 'Хабаровск', 'Томск'],
    'Баланс на счете': [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 98000,
                       250000, 210000, 135000, 155000, 125000, 180000, 275000, 320000, 105000, 90000],
    'Кредитная история': ['Хорошая', 'Средняя', 'Плохая', 'Хорошая', 'Средняя', 'Отличная', 'Средняя', 'Хорошая', 'Плохая', 'Средняя',
                           'Хорошая', 'Отличная', 'Средняя', 'Хорошая', 'Средняя', 'Плохая', 'Отличная', 'Хорошая', 'Средняя', 'Плохая']
}

df = pd.DataFrame(data)

print("Исходный DataFrame:")
print(df)
print("\n")

print("1. Клиенты из 'Москва' или 'Санкт-Петербург' (.isin())")

cities = ['Москва', 'Санкт-Петербург']
df_city_filter = df[df['Город'].isin(cities)]

print(f"Клиенты из городов: {cities}")
print(df_city_filter)
print("\n")

print("2. Клиенты с балансом от 100000 до 250000 (.between())")

df_balance_filter = df[df['Баланс на счете'].between(100000, 250000)]

print("Клиенты с балансом от 100000 до 250000:")
print(df_balance_filter)
print("\n")

print("3. Клиенты с 'Хорошей' кредитной историей и балансом > 150000 (.query())")

df_query_filter = df.query("'Кредитная история' == 'Хорошая' and 'Баланс на счете' > 150000")

print("Клиенты с хорошей кредитной историей и балансом > 150000:")
print(df_query_filter)
print("\n")

```

Рисунок 40. Код 6 задания

Исходный DataFrame:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
4	5	Сергей	29	Екатеринбург	95000	Средняя
5	6	Анна	50	Воронеж	300000	Отличная
6	7	Дмитрий	31	Челябинск	140000	Средняя
7	8	Елена	40	Краснодар	175000	Хорошая
8	9	Виктор	28	Ростов-на-Дону	110000	Плохая
9	10	Алиса	33	Уфа	98000	Средняя
10	11	Павел	46	Омск	250000	Хорошая
11	12	Светлана	37	Пермь	210000	Отличная
12	13	Роман	41	Тюмень	135000	Средняя
13	14	Татьяна	25	Саратов	155000	Хорошая
14	15	Николай	39	Самара	125000	Средняя
15	16	Валерия	42	Волгоград	180000	Плохая
16	17	Григорий	49	Барнаул	275000	Отличная
17	18	Юлия	50	Иркутск	320000	Хорошая
18	19	Степан	30	Хабаровск	105000	Средняя
19	20	Василиса	35	Томск	90000	Плохая

1. Клиенты из 'Москва' или 'Санкт-Петербург' (.isin())

Клиенты из городов: ['Москва', 'Санкт-Петербург']

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя

2. Клиенты с балансом от 100000 до 250000 (.between())

Клиенты с балансом от 100000 до 250000:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34	Москва	120000	Хорошая
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
6	7	Дмитрий	31	Челябинск	140000	Средняя
7	8	Елена	40	Краснодар	175000	Хорошая
8	9	Виктор	28	Ростов-на-Дону	110000	Плохая
10	11	Павел	46	Омск	250000	Хорошая
11	12	Светлана	37	Пермь	210000	Отличная
12	13	Роман	41	Тюмень	135000	Средняя
13	14	Татьяна	25	Саратов	155000	Хорошая
14	15	Николай	39	Самара	125000	Средняя
15	16	Валерия	42	Волгоград	180000	Плохая
18	19	Степан	30	Хабаровск	105000	Средняя

3. Клиенты с 'Хорошей' кредитной историей и балансом > 150000 (.query())

Клиенты с хорошей кредитной историей и балансом > 150000:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
3	4	Мария	38	Новосибирск	200000	Хорошая
7	8	Елена	40	Краснодар	175000	Хорошая
10	11	Павел	46	Омск	250000	Хорошая
13	14	Татьяна	25	Саратов	155000	Хорошая
17	18	Юлия	50	Иркутск	320000	Хорошая

Рисунок 41. Результат 6 задания

7. Подсчет значений (count, value_counts, nunique)

Используя Таблицу 2:

1. Подсчитайте количество **непустых значений** в каждом столбце (`.count()`).
2. Определите **частоту встречаемости значений** в "Город" (`.value_counts()`).
3. Найдите **количество уникальных значений** в "Город", "Возраст", "Баланс на счете" (`.nunique()`).

Выведите результаты.

Рисунок 42. Задание 7

```
import pandas as pd

pd.set_option('display.width', None)
pd.set_option('display.max_columns', None)

data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'Анна', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
            'Павел', 'Светлана', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василис'],
    'Возраст': [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    'Город': ['Москва', 'Санкт-Петербург', 'Казань', 'Новосибирск', 'Екатеринбург', 'Воронеж', 'Челябинск', 'Краснодар', 'Ростов-на-Дону', 'Уфа',
              'Омск', 'Пермь', 'Тюмень', 'Саратов', 'Самара', 'Волгоград', 'Барнаул', 'Иркутск', 'Хабаровск', 'Томск'],
    'Баланс на счете': [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 90000,
                       250000, 210000, 135000, 155000, 125000, 180000, 275000, 320000, 105000, 90000],
    'Кредитная история': ['Хорошая', 'Средняя', 'Плохая', 'Хорошая', 'Средняя', 'Отличная', 'Средняя', 'Хорошая', 'Плохая', 'Средняя',
                           'Хорошая', 'Отличная', 'Средняя', 'Хорошая', 'Средняя', 'Плохая', 'Отличная', 'Хорошая', 'Средняя', 'Плохая']
}

df = pd.DataFrame(data)

print("Исходный DataFrame:")

print(df)
print("\n")

print("1. Количество непустых значений в каждом столбце (.count())")

non_null = df.count()
print("Количество непустых значений по столбцам:")
print(non_null)
print("\n")

print("2. Частота встречаемости городов (.value_counts())")

city_counts = df['Город'].value_counts()
print("Количество клиентов по городам:")
print(city_counts)
print("\n")

print("3. Количество уникальных значений в столбцах (.nunique())")

unique = df[['Город', 'Возраст', 'Баланс на счете']].nunique()
print("Количество уникальных значений:")
print(unique)
```

Рисунок 43. Код 7 задания

```

Исходный DataFrame:
   ID  Имя  Возраст  Город  Баланс на счете  Кредитная история
0   1  Иван    34      Москва      120000      Хорошая
1   2  Ольга    27  Санкт-Петербург      80000      Средняя
2   3  Алексей   45      Казань      150000      Плохая
3   4  Мария    38  Новосибирск      200000      Хорошая
4   5  Сергей    29  Екатеринбург      95000      Средняя
5   6  Анна     50      Воронеж      300000      Отличная
6   7  Дмитрий   31      Челябинск      140000      Средняя
7   8  Елена    40      Краснодар      175000      Хорошая
8   9  Виктор    28  Ростов-на-Дону      110000      Плохая
9  10  Алиса    33        Уфа      98000      Средняя
10  11  Павел    46        Омск      250000      Хорошая
11  12  Светлана  37        Пермь      210000      Отличная
12  13  Роман    41        Тюмень      135000      Средняя
13  14  Татьяна   25      Саратов      155000      Хорошая
14  15  Николай   39      Самара      125000      Средняя
15  16  Валерия   42      Волгоград      180000      Плохая
16  17  Григорий  49      Барнаул      275000      Отличная
17  18  Юлия     50      Иркутск      320000      Хорошая
18  19  Степан   30      Хабаровск      105000      Средняя
19  20  Василиса  35        Томск      90000      Плохая

```

1. Количество непустых значений в каждом столбце (.count())

Количество непустых значений по столбцам:

```

ID      20
Имя     20
Возраст 20
Город   20
Баланс на счете 20
Кредитная история 20
dtype: int64

```

2. Частота встречаемости городов (.value_counts())

Количество клиентов по городам:

```

Город
Москва      1
Санкт-Петербург  1
Хабаровск    1
Иркутск      1
Барнаул      1
Волгоград    1
Самара       1
Саратов      1
Тюмень       1
Пермь        1
Омск         1
Уфа          1
Ростов-на-Дону  1
Краснодар    1
Челябинск    1
Воронеж      1
Екатеринбург  1
Новосибирск  1
Казань       1
Томск        1
Name: count, dtype: int64

```

3. Количество уникальных значений в столбцах (.nunique())

Количество уникальных значений:

```

Город      20
Возраст    19
Баланс на счете 20
dtype: int64

```

Рисунок 44. Результат 7 задания

8. Обнаружение пропусков (`isna` , `notna`)

Используя Таблицу 3:

1. Подсчитайте количество NaN в каждом столбце (`.isna().sum()`).
2. Подсчитайте количество заполненных значений в каждом столбце (`.notna().sum()`).
3. Выведите DataFrame , оставив только строки, где нет пропущенных значений.

Объясните разницу между `.isna()` и `.notna()` .

Рисунок 45. Задание 8

```
import pandas as pd
import numpy as np

pd.set_option('display.width', None)
pd.set_option('display.max_columns', None)

data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    'Имя': ['Иван', 'Ольга', 'Алексей', 'Мария', 'Сергей', 'None', 'Дмитрий', 'Елена', 'Виктор', 'Алиса',
           'Павел', 'None', 'Роман', 'Татьяна', 'Николай', 'Валерия', 'Григорий', 'Юлия', 'Степан', 'Василиса'],
    'Возраст': [34.0, 27.0, 'None', 38.0, 29.0, 50.0, 31.0, 40.0, 'None', 33.0,
               46.0, 37.0, 41.0, 25.0, 'None', 42.0, 49.0, 'None', 30.0, 35.0],
    'Город': ['Москва', 'Санкт-Петербург', 'Казань', 'None', 'Екатеринбург', 'Воронеж', 'None', 'Краснодар', 'Ростов-на-Дону', 'Уфа',
              'Омск', 'Пермь', 'None', 'Саратов', 'Самара', 'None', 'Барнаул', 'Иркутск', 'Хабаровск', 'Томск'],
    'Баланс на счете': [120000.0, 'None', 150000.0, 200000.0, 'None', 300000.0, 140000.0, 175000.0, 110000.0, 'None',
                       250000.0, 210000.0, 135000.0, 155000.0, 125000.0, 'None', 275000.0, 320000.0, 105000.0, 90000.0],
    'Кредитная история': ['Хорошая', 'Средняя', 'Плохая', 'Хорошая', 'None', 'Отличная', 'Средняя', 'Хорошая', 'None', 'Средняя',
                           'Хорошая', 'Отличная', 'Средняя', 'None', 'Средняя', 'Плохая', 'Отличная', 'Хорошая', 'Средняя', 'Плохая']
}

df = pd.DataFrame(data)

print("Исходный DataFrame (Таблица 3 - с пропусками):")
print(df)
print("\n")

print("1. Количество пропусков (NaN) в каждом столбце (.isna().sum())")

nan_counts = df.isna().sum()
print("Количество пропусков по столбцам:")
print(nan_counts)
print("\n")

print("2. Количество заполненных значений в каждом столбце (.notna().sum())")

not_nan_counts = df.notna().sum()
print("Количество заполненных значений по столбцам:")
print(not_nan_counts)
print("\n")

print("3. DataFrame без пропущенных значений (только полные строки)")

df_clean = df[df.notna().all(axis=1)]
print("DataFrame без строк с пропусками:")
print(df_clean)
print("\n")

print("Разница между .isna() И .notna():")
print("""
.isna() - обнаруживает пропуски
.notna() - обнаруживает заполненные значения
""")
```

Рисунок 46. Код задания 8

Исходный DataFrame (Таблица 3 - с пропусками):

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34.0	Москва	120000.0	Хорошая
1	2	Ольга	27.0	Санкт-Петербург	NaN	Средняя
2	3	Алексей	NaN	Казань	150000.0	Плохая
3	4	Мария	38.0	None	200000.0	Хорошая
4	5	Сергей	29.0	Екатеринбург	NaN	None
5	6	None	50.0	Воронеж	300000.0	Отличная
6	7	Дмитрий	31.0	None	140000.0	Средняя
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
8	9	Виктор	NaN	Ростов-на-Дону	110000.0	None
9	10	Алиса	33.0	Уфа	NaN	Средняя
10	11	Павел	46.0	Омск	250000.0	Хорошая
11	12	None	37.0	Пермь	210000.0	Отличная
12	13	Роман	41.0	None	135000.0	Средняя
13	14	Татьяна	25.0	Саратов	155000.0	None
14	15	Николай	NaN	Самара	125000.0	Средняя
15	16	Валерия	42.0	None	NaN	Плохая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
17	18	Юлия	NaN	Иркутск	320000.0	Хорошая
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Василиса	35.0	Томск	90000.0	Плохая

1. Количество пропусков (NaN) в каждом столбце (.isna().sum())

Количество пропусков по столбцам:

ID	0
Имя	2
Возраст	4
Город	4
Баланс на счете	4
Кредитная история	3
dtype:	int64

2. Количество заполненных значений в каждом столбце (.notna().sum())

Количество заполненных значений по столбцам:

ID	20
Имя	18
Возраст	16
Город	16
Баланс на счете	16
Кредитная история	17
dtype:	int64

3. DataFrame без пропущенных значений (только полные строки)

DataFrame без строк с пропусками:

	ID	Имя	Возраст	Город	Баланс на счете	Кредитная история
0	1	Иван	34.0	Москва	120000.0	Хорошая
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
10	11	Павел	46.0	Омск	250000.0	Хорошая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Василиса	35.0	Томск	90000.0	Плохая

Разница между .isna() и .notna():

.isna() - обнаруживает пропуски
 .notna() - обнаруживает заполненные значениями

Рисунок 47. Результат задания 8

6. Было выполнено индивидуальное задание.

Использовать DataFrame, содержащий следующие колонки: название пункта назначения; номер поезда; время отправления. Написать программу, выполняющую следующие действия: ввод с клавиатуры данных и добавление строк в DataFrame; записи должны быть размещены в алфавитном порядке по названиям пунктов назначения; вывод на экран информации о поездах,

отправляющихся после введенного с клавиатуры времени; если таких поездов нет, выдать на дисплей соответствующее сообщение.

```
import pandas as pd
from datetime import datetime
import os

FILE = "trains.parquet"

if os.path.exists(FILE):
    df = pd.read_parquet(FILE)
else:
    df = pd.DataFrame(columns=['Пункт', 'Номер', 'Время'])

while True:
    print("1 - Добавить поезд")
    print("2 - Показать все")
    print("3 - Найти после времени")
    print("4 - Удалить")
    print("5 - Выйти")

    choice = input("Выберите: ")

    if choice == '1':
        punkt = input("Пункт: ")
        nomer = input("Номер: ")
        vrema = input("Время (ЧЧ:ММ): ")

        try:
            datetime.strptime(vrema, "%H:%M")
        except:
            print("Неверное время")
            continue

        new_row = pd.DataFrame({
            'Пункт': [punkt],
            'Номер': [nomer],
            'Время': [vrema]
        })
        df = pd.concat([df, new_row], ignore_index=True)
        df = df.sort_values('Пункт').reset_index(drop=True)
        df.to_parquet(FILE, index=False)
        print("Добавлено!")

    elif choice == '2':
        if df.empty:
            print("Список пуст")
        else:
            print("\n", df.to_string(index=False))

    elif choice == '3':
        if df.empty:
            print("Список пуст")
            continue

        vrema = input("Время (ЧЧ:ММ): ")
        try:
            search_time = datetime.strptime(vrema, "%H:%M").time()
        except:
            print("Неверное время")
            continue

        found = []
        for i, row in df.iterrows():
            t = datetime.strptime(row['Время'], "%H:%M").time()
            if t > search_time:
                found.append(row)
```

Рисунок 48. Индивидуальное задание (1)

```

if found:
    result = pd.DataFrame(found)
    print(f"\n Поезда после {vremya}:")
    print(result.to_string(index=False))
else:
    print(f"Нет поездов после {vremya}")

elif choice == '4':
    if df.empty:
        print("Список пуст")
        continue

    print("Колонки:", list(df.columns))
    col = input("Колонка: ")
    val = input("Значение: ")

    before = len(df)
    df = df[df[col] != val]
    deleted = before - len(df)

    if deleted > 0:
        df.to_parquet(FILE, index=False)
        print(f"Удалено {deleted} строк")
    else:
        print("Ничего не найдено")

elif choice == '5':
    df.to_parquet(FILE, index=False)
    break

else:
    print("Неверный выбор")

```

```

1 - Добавить поезд
2 - Показать все
3 - Найти после времени
4 - Удалить
5 - Выйти
Выберите: 1
Пункт: Став
Номер: 7
Время (ЧЧ:ММ): 12:00
Добавлено!
1 - Добавить поезд
2 - Показать все
3 - Найти после времени
4 - Удалить
5 - Выйти
Выберите: 2

Пункт Номер Время
Став      7 12:00
1 - Добавить поезд
2 - Показать все
3 - Найти после времени
4 - Удалить
5 - Выйти
Выберите: 5

```

Рисунок 49. Индивидуальное задание (2)

Контрольные вопросы:

1. Создать DataFrame из словаря списков — передать словарь, где ключи — названия столбцов, а значения — списки данных, в конструктор `pd.DataFrame()`.

2. Отличие списка словарей от словаря списков — в первом случае каждая запись — это отдельный словарь (строка), во втором — столбцы

задаются списками значений. Первый удобен для построчного добавления, второй — для столбцового.

3. Из массива NumPy — передать массив в `pd.DataFrame()` и при желании указать имена столбцов через параметр `columns`.

4. Загрузить CSV с разделителем ; — в `pd.read_csv()` указать параметр `delimiter=';'` или `sep=';'`.

5. Из Excel с выбором листа — использовать `pd.read_excel()` с параметром `sheet_name`, куда передать имя листа или его номер.

6. Отличие JSON от Parquet — JSON — текстовый, медленный, но читаемый; Parquet — бинарный, сжатый, быстрый и сохраняет типы, лучше для больших объёмов.

7. Проверить типы данных — вызвать атрибут `.dtypes` у `DataFrame`.

8. Узнать размер — использовать атрибут `.shape`, который возвращает кортеж (строки, столбцы).

9. Разница `.loc[]` и `.iloc[]` — `.loc[]` работает по меткам индексов и столбцов, а `.iloc[]` — по целочисленным позициям.

10. Третья строка, второй столбец через `.iloc[]` — написать `df.iloc[2, 1]`, так как нумерация начинается с нуля.

11. Строка с индексом "Мария" — использовать `df.loc['Мария']`, если индексом является имя.

12. Отличие `.at[]` от `.loc[]` — `.at[]` предназначен для быстрого доступа к одному конкретному значению по меткам, а `.loc[]` может выбирать целые диапазоны.

13. Когда `.iat[]` быстрее `.iloc[]` — при обращении к одному скалярному элементу по целочисленному индексу, так как он оптимизирован именно для этого случая.

14. Выбрать строки с городами "Москва" или "СПб" через `.isin()` — применить маску `df['Город'].isin(['Москва', 'СПб'])` и передать её в фильтр.

15. Отфильтровать возраст от 25 до 35 через `.between()` — использовать `df['Возраст'].between(25, 35)` как условие фильтрации.

16. Разница `.query()` и `.loc[]` — `.query()` принимает строковое выражение, удобно для сложных условий, а `.loc[]` работает с булевой маской и обычно быстрее на больших данных.

17. Использовать переменные в `.query()` — внутри строки запроса обратиться к переменной через префикс.

18. Количество пропусков в каждом столбце — сложить результаты `.isna()` по столбцам с помощью `.sum()`.

19. Разница `.isna()` и `.notna()` — первая показывает `True` для пропусков, вторая — для заполненных ячеек, это полные противоположности.

20. Вывести строки без пропусков — применить метод `.dropna()` без дополнительных параметров.

21. Добавить столбец с фиксированным значением — просто присвоить новому столбцу строку или число, и оно распространится на все строки.

22. Добавить строку через `.loc[]` — обратиться к новому индексу (например, следующему числовому) и присвоить ему список значений.

23. Удалить столбец — использовать `.drop()` с указанием имени столбца и `axis=1`, либо с параметром `inplace=True` для изменения текущего `DataFrame`.

24. Удалить строки с любым `NaN` — вызвать `.dropna()` с `axis=0` (по умолчанию) и `how='any'`.

25. Удалить столбцы с любым `NaN` — в `.dropna()` указать `axis=1` и `how='any'`.

26. Количество непустых значений в столбцах — использовать метод `.count()`, который игнорирует пропуски.

27. Отличие `.value_counts()` от `.nunique()` — первая показывает частоту каждого уникального значения, вторая — только общее количество уникальных значений.

28. Частота значений в столбце "Город" — применить `.value_counts()` к этому столбцу.

29. Почему `display()` лучше `print()` в Jupyter — `display()` визуализирует данные в виде красивой интерактивной таблицы, а `print()` выводит лишь урезанный текстовый вариант.

30. Изменить лимит отображаемых строк — настроить параметр `display.max_rows` через `pd.set_option()`, указав нужное число или `None` для показа всех строк.